1. قلب البروجكت: `CrawlerEngine.java` (المقاول والعمال)

ده الكلاس المعلم اللي بيمشي الشغلانة كلها، وفيه كل شغل مادة الـ Parallel Programming.

- **طابور الشغل (queue):** ده معمول بنوع اسمه `LinkedBlockingQueue`. "مؤمن" عشان ده طابور "ليه؟ لما تلاقي لينكات جديدة في موقع، بنرميها في الطابور ده، والعمال يسحبوا منه واحد ورا التاني (Thread-safe). من غير ما يتخانقوا أو يسحبوا نفس اللينك.
- **دفتر الزيارات (visitedUrls):** معمول بنوع اسمه `ConcurrentHashMap`. ده الدفتر اللي بنسجل فيه أي لينك دخلناه عشان منرجعلوش تاني. ميزته إنه بيسمح لكذا عامل يكتبوا فيه في نفس اللحظة من غير ما البرنامج يكراش.
- **العداد السحري (pageCount):** معمول بنوع `AtomicInteger`. كلمة `Atomic` ده بيعد إحنا حملنا كام صفحة. دي معناها إن العداد ده محمي، مستحيل عاملين يزودوه في نفس الميكرو-ثانية ويضيعوا عدَّة مننا.
- **دالة `runParallel()`:** هنا بنجيب المقاول `ExecutorService` ونقوله "هاتلي 6 عمال (Threads)". المقاول بياخد لستة لينكات من الطابور (`drainTo`)، ويوزعها على العمال في نفس اللحظة باستخدام `executor.submit()`. العمال ينطلقوا في النت مع بعض، وكل ما يخلصوا موقع يرجعولنا بالنتيجة (`Future.get`).
- **دالة `crawlPageWithRetry(task, attempt)`:** دي اللي العامل بينفذها:
  1. **ينام شوية:** `Thread.sleep(politenessDelayMs)` عشان سيرفرات الناس متعملناش حظر لو خبطنا عليهم بسرعة.
  2. **يخبط على الموقع:** بيستخدم مكتبة `Jsoup` عشان يحمل الصفحة.
  3. **يستخرج الداتا (Parsing):** بيجيب العنوان، الوصف، ويدور على كل اللينكات `<a href>` اللي في الصفحة.
  4. **يضيف للشغل:** `queue.offer` أي لينك جديد بيلاقيه، بيحطه في الطابور عشان العمال التانيين يروحوله.
  5. **الخطة ب (Retry Logic):** لو النت قطع أو الموقع وقع ورمى `IOException`، العامل بيبص على الـ `attempt` (المحاولات). لو دي أول مرة، بيقول "هحاول تاني" وينادي الدالة من جديد. لو فشل تاني، بيكتب "ERROR" ويرجع.

---

2. أمناء المخازن: `SqliteStorage` و`CsvStorage` و`JsonStorage`

دول الكلاسات المسؤولة عن حفظ الداتا اللي العمال جابوها.

- **عسكري المرور (synchronized):** في أي كلاس من دول، هتلاقي قبلها كلمة `synchronized`. لو بصيت على دالة `save()`
- **ليه؟ (Race Condition Prevention):** تخيل 6 عمال خلصوا 6 صفحات في نفس اللحظة، وجريوا كلهم بتعملهم "باب دوار"، بتجبرهم يقفوا طابور، ومفيش عامل يقدر يكتب في الداتا بيز إلا لما اللي قبله يخلص ويقفل الباب وراه. يعني بنحمل من النت بالتوازي، بس بنكتب في الهارد بالتسلسل عشان الأمان. النتيجة؟ الملف هيبوظ والكلام هيدخل في بعضه. كلمة `synchronized` (`SQLite`) أو الداتا بيز (`CSV`) يكتبوا في ملف الإكسيل

---

3. الاستمارات والإعدادات: `CrawledPage` و`CrawlerConfig`

دول الكلاسات اللي بتشيل الداتا (Models).

- **`CrawledPage`:** دي الاستمارة الفاضية. أي عامل بيخلص موقع، بيملا الاستمارة دي (اسم الموقع، اللينكات اللي فيه، حالته 200 ولا 404، وخد وقت قد إيه).
- **`CrawlerConfig`:** ديزاين باترن محترم جداً). ده بيلم إعدادات (`Builder Pattern`ده كلاس معمول بـ `CrawlerEngine`اليوزر كلها (كام عامل، أقصى عمق، اللينك الأساسي) ويبني بيها كائن واحد متبرشم نبعته لـ عشان يشتغل بيه.

---

4. واجهة العرض: `MainController.java` (الشاشة)

بكود المحرك بتاعنا (JavaFX) ده الكلاس اللي بيربط زراير الشاشة.

- **زرار الـ Start:** `CrawlerConfig`لما اليوزر بيدوس عليه، الكلاس ده بياخد الأرقام من الشاشة، يبني الـ ، `CrawlerEngine`ويقوله `start()`ويعمل كائن جديد من .
- **سر الصنعة (`Platform.runLater`):** في البرمجة، الشاشة (GUI) ليها عامل مخصص بتاعها اسمه (Main Thread). ممنوع منعاً باتاً أي عامل تاني (من بتوع النت) يلمس الشاشة أو يكتب فيها. عشان كده، لما العمال بتوعنا بيخلصوا صفحة وعايزين يكتبوا في الجدول اللي في الشاشة، بيبعتوا رسالة لـ `Platform.runLater()`، ودي بتوصل الداتا لشاشة العرض بأمان من غير ما البرنامج يهنج.
- **المقارنة (Benchmark Mode):** الكلاس ده بيقرأ إنت كاتب كام في خانة الـ Threads. ، بيشغل 1لو كاتب `Parallel`، بيشغل 6ويحسب الوقت. لو كاتب `Sequential`البرنامج ويحسب الوقت.

1. رسمة الـ Use Case Diagram (قائمة الطلبات)

- **ببساطة:** الرسمة دي عاملة زي "المنيو" بتاع المطعم. بتوريك إنت (User كـ) واقف على جنب، والنت (Web Server) واقف على الجنب التاني، وفي النص كل الحاجات اللي تقدر تعملها في البرنامج.
- **فايدتها:** تختار صيغة الحفظStop ، تعملStart بتوضح إنت تقدر تدوس ، وتشوف النتايج. (CSV/JSON) ملهاش دعوة خالص الكود مكتوب إزاي، دي بس بتعرف الدكتور إن البرنامج بتاعك فيه كل الميزات المطلوبة.

2. رسمة الـ Class Diagram (خريطة الكود)

- **ببساطة:** دي "الخريطة الهندسية" بتاعة البروجكت بتاعك. المربعات اللي فيها دي هي ملفات الجافا بتاعتك (زي `CrawledPage`و `CrawlerEngine`).
- **فايدتها:** بتوريك كل كلاس جواه متغيرات إيه ودوال (Functions) إيه، ومين بيكلم مين. دي بتثبت إنك منظم الكود بتاعك صح ومقسمه باحترافية.

3. رسمة الـ Sequence Diagram (تسلسل الأحداث)

- **ببساطة:** لحد ما "Start" دي بتحكي "قصة" بالترتيب الزمني. بتبدأ من فوق لتحت، من أول ما بتدوس على زرار البرنامج يخلص.
- **فايدتها:** بتوضح مسار الرسائل؛ الشاشة كلمت المحرك، فالمحرك جاب العمال (Thread Pool) ، والعمال راحوا (Storage). ، وبعدين جابوا الداتا ودوها للمخازن(Web Server) كلموا النت بتثبت إنك فاهم مين بيشتغل الأول ومين بيستنى التاني.

4. رسمة الـ Activity Diagram (خط سير العامل)

- **ببساطة:** "دي بقى بتركز مع "عامل واحد بس (واحد Thread) وماشية وراه خطوة بخطوة وهو بيجيب الصفحة.
- **فايدتها:** بتشرح "اللوجيك" والقرارات. يعني العامل بيستنى شوية، وبعدين يحاول يحمل الصفحة، **ولو النت قطع** ، وبعدين يطلع اللينكات ويسجلها. دي أهم رسمة بتشرح الشغل (Retry Logic دي الـ) بيعمل إيه؟ بيجرب تاني الفعلي جوه الكود.

5. رسمة الـ Component Diagram (النظرة من الطيارة)

- **ببساطة:** دي بتجيب البروجكت من فوق أوي، وبتقسمه لـ 3 حتت أو "طبقات" كبار:
  1. طبقة الشاشة (Presentation Layer).
  2. طبقة المحرك والعمال اللي بيشتغلوا بالتوازي (Core Engine Layer).
  3. طبقة الحفظ والداتا بيز (Storage Layer).
- **فايدتها:** بتفهم اللي بيشوفها إنك مش عامل كود سايح على بعضه، لأ إنت مقسم البرنامج لمعمارية نضيفة (Architecture).

6. رسمة الـ State Diagram (حالات البرنامج)

- **ببساطة:** بتوضح "مزاج" أو "حالة" البرنامج وإزاي بيتنقل من حالة للتانية.
- **فايدتها:** ولما بتدوس ستارت بيدخل في حالة(READY/جاهز) بتقول إن البرنامج بيبدأ وهو ، ولما يخلص (Parallel) أو وضع سريع (Sequential) ، وممكن يشتغل بوضع بطيء(CRAWLING/شغال) ، بتوضح دورة حياة البرنامج من الألف للياء. خالص بيروح لحالة (DONE/خلص).